Introduction to NLP & Text Analytics

Text Pre-processing · Feature Extraction · Embeddings · Classification

NLP Course · PES University

Department of Computer Science & Engineering

Session 1

Lecture Outline

Natural Language Processing

Definition

NLP is a sub-field of Artificial Intelligence that enables computers to **understand, interpret, and generate** human language in a useful way. [8]

Why is NLP hard?

- **Ambiguity** — "I saw the man with a telescope"
- **Context-dependence** — same word, different meaning
- **Coreference** — "She said *she* was tired"
- **Figurative language** — irony, metaphor
- **World knowledge** — implicit

Difficulty scale (1–10)

Task	Rule	ML
Tokenization	2	1
POS Tagging	4	2
NER	5	3
Sentiment	6	4
Coreference	9	7
Reasoning	10	9

NLP Application Landscape

Understanding

- Sentiment Analysis
- Named Entity Recognition
- Syntactic Parsing
- Coreference Resolution
- Information Extraction

Generation

- Machine Translation
- Summarisation
- Dialogue / Chatbots
- Text-to-SQL
- Story generation

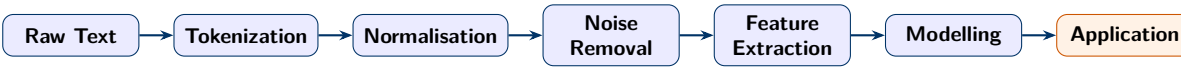
Hybrid

- Question Answering
- Code Generation
- Document Q&A (RAG)
- Caption generation
- Instruction following

Industry Uptake

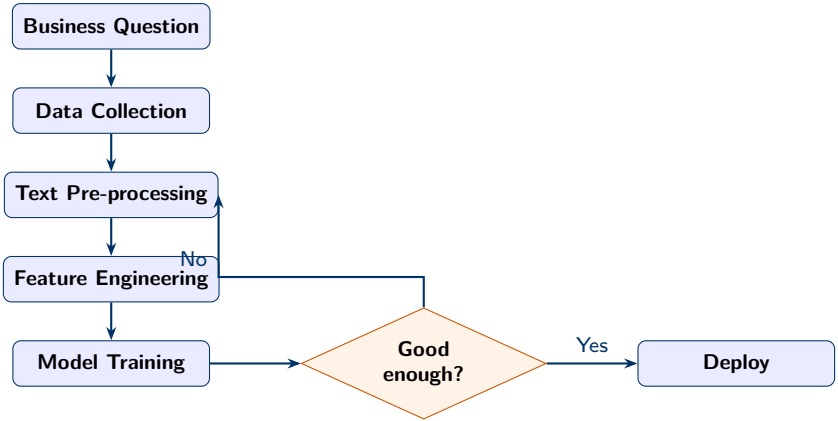
LLM-based NLP products (ChatGPT, Copilot, Gemini) reached >100 M users within months of release — the fastest technology adoption on record. [13]

The Standard NLP Pipeline



Stage	Purpose	Example tools
Tokenization	Split text into atomic units	NLTK, spaCy, HuggingFace
Normalisation	Lowercase, expand contractions	regex, `unicodedata`
Noise Removal	Strip HTML/URLs/punctuation	BeautifulSoup, regex
Feature Extraction	Convert text to numeric vectors	TF-IDF, Word2Vec, BERT
Modelling	Train statistical/neural model	scikit-learn, PyTorch
Application	Deploy to downstream product	Flask, FastAPI

Text Analytics Lifecycle



Key Insight

The pipeline is **iterative**, not linear. The "good enough?" gate drives repeated cycles of

NLTK vs spaCy

NLTK [1]

Created 2001 at U. Penn by Bird & Loper.

- 80+ bundled corpora (WordNet, Reuters, Brown)
- Rich stemmer suite (Porter, Lancaster, Snowball)
- Ideal for **teaching** and linguistic research
- Explicit, step-by-step API

spaCy [6]

Created 2015 by Honnibal & Montani (Explosion AI).

- Industrial-strength neural pipeline
- Pre-trained models for 60+ languages
- Dependency parsing, NER, coref out-of-the-box
- **Production** oriented; 10–50× faster

Feature	**NLTK**	**spaCy**
POS Tagging	Rule + HMM	Neural (CNN)
Stemming	Porter/Lancaster/Snowball	—
Lemmatization	WordNet lookup	Lookup + rules
NER	MaxEnt/chunker	Neural (transition)
Word Vectors	via Gensim	Built-in (GloVe)
Speed	Moderate	Very fast

Pre-processing Pipeline — Step by Step

Step	What it does	Caution
Lowercasing	Unify “Apple” and “apple”	Skip for NER (case = signal)
Spell correction	Fix “recieve” → “receive”	Slow; may corrupt jargon
URL removal	Strip `https://...`	Keep for URL classification
HTML stripping	Remove `<p>`, `<div>`	Use BeautifulSoup
Unicode normalise	é→e, "→ " → ASCII	Needed for non-English
Punctuation strip	Remove `!.,;?`	Keep for sentiment (!)
Tokenize	Split into word units	Choose tokenizer carefully
Stopword removal	Drop “the”, “is”, “at”	Domain stopwords differ
Stem/Lemmatize	“running” → “run”	Over-stemming merges meaning

Pitfall

Removing stopwords before **sentiment analysis** degrades accuracy — words like “not”, “never”, “hardly” are grammatically function words but are semantically critical.

Tokenizers Compared

```
from nltk.tokenize import (word_tokenize,
                           TweetTokenizer)
tweet = "OMG! Can't believe #NLP is SO
    cool"
# Whitespace split
ws      = tweet.split()                # 8
# NLTK word_tokenize
nltk_t = word_tokenize(tweet)          # 11
# TweetTokenizer
tt      = TweetTokenizer().tokenize(tweet)
     # 8
```

Why token counts differ:

- **Whitespace**: splits on spaces only; "Can't" stays whole; no URL handling
- **word_tokenize**: Penn TB rules; splits contractions ("Can't"→["Ca","n't"]); '!' separate
- **TweetTokenizer**: preserves #tags, @handles, emoji, URLs as single tokens

Rule of thumb: match tokenizer to domain — TweetTokenizer for social media, word_tokenize for prose.

Stemming vs. Lemmatization [10]

Stemming

Applies suffix-stripping **rules** mechanically.

- **Porter** (1980) — 5-step algorithm, most common [15]
- **Lancaster** — more aggressive; faster convergence
- **Snowball** — multi-language, improved Porter
- Output may not be a real word: `studies → studi`

Lemmatization

Uses POS tag + **dictionary** lookup.

- Requires POS tagging as prerequisite
- Always returns a real dictionary form
- Handles irregular forms: `better → good`
- WordNet [12] in NLTK; lookup tables in spaCy

Word	Porter	Lancaster	WordNet Lemma
studies	studi	study	study
running	run	run	run
better	better	bet	**good**
geese	gees	gees	**goose**

Bag-of-Words & TF-IDF

Bag-of-Words (BoW) [18]

1. Build vocabulary V (all unique tokens)
2. Represent each document as a count vector of length $|V|$
3. Ignores word order; very sparse

Problem: "the", "is" get large counts but carry no meaning.

TF-IDF [7, 17]

$$\text{TF-IDF}(t,d) = \underbrace{\frac{f_{t,d}}{\sum_k f_{k,d}}}_{\text{TF}} \times \underbrace{\log \frac{N+1}{n_t+1} + 1}_{\text{IDF}}$$

- $f_{t,d}$ — raw frequency of t in d
- N — total documents; n_t — docs containing t
- High IDF $\Rightarrow$ rare, discriminating term
- sklearn uses `sublinear_tf`: $1 + \log f_{t,d}$ to compress outliers

TF-IDF — Worked Example

Mini corpus (4 docs, N=4):

- D1: "the cat sat on the mat" D2: "the dog sat on the log"
- D3: "the cat is a lazy animal" D4: "the dog is a brown animal"

Term	n_t (df)	IDF $= \log(5/n_t) + 1$	TF in D1	TF-IDF (D1)
the	4	$\log(1.25) + 1 = 1.22$	2/6=0.33	0.41
cat	2	$\log(2.5) + 1 = 1.92$	1/6=0.17	0.32
sat	2	$\log(2.5) + 1 = 1.92$	1/6=0.17	0.32
mat	1	$\log(5) + 1 = 2.61$	1/6=0.17	0.44
dog	2	$\log(2.5) + 1 = 1.92$	0	0

Key Insight

"mat" gets the highest TF-IDF in D1 — it appears only there. **"the"** has high TF but near-zero IDF (appears in all docs), so it contributes little discrimination. TF-IDF automatically acts as a soft stopword filter.

N-grams [19]

Definition

An **n-gram** is a contiguous sequence of n tokens. Captures short-range phrase context invisible to unigrams.

Examples from "the quick brown fox":

- Unigram: [the, quick, brown, fox]
- Bigram: [the quick, quick brown, brown fox]
- Trigram: [the quick brown, quick brown fox]

Bigram Language Model:

$$P(w_n \mid w_{n-1}) = \frac{C(w_{n-1}, w_n)}{C(w_{n-1})}$$

Laplace smoothing prevents zero probabilities:

$$P_{\mathrm{L}}(w_n \mid w_{n-1}) = \frac{C(w_{n-1}, w_n) + 1}{C(w_{n-1}) + |V|}$$

Practical Note

Part-of-Speech Tagging

Assigns grammatical labels (Noun, Verb, Adjective . . .) to each token.

Penn Treebank Tags (common subset):

Tag	Meaning	Example	Tag	Meaning	Example
NN	Noun, singular	dog	VB	Verb, base	run
NNS	Noun, plural	dogs	VBZ	Verb, 3sg	runs
NNP	Proper noun	London	VBD	Verb, past	ran
JJ	Adjective	quick	IN	Preposition	over
RB	Adverb	quickly	DT	Determiner	the

Why POS matters

Required by lemmatization ("studies" as noun → "study"; "studies" as verb → "study"), syntactic parsing, NER, and information extraction pipelines.

POS Tagging — Three Generations of Models

1. Rule-based (1980s)

- Hand-written patterns
- "word ends in -ing after aux. → VBG"
- Fast, transparent
- Brittle on novel text

2. HMM (1990s) [16]

- Emission: $P(\text{word}|\text{tag})$
- Transition: $P(\text{tag}_i|\text{tag}_{i-1})$
- **Markov assumption**: tag depends only on previous tag
- Viterbi decoding: $O(nT^2)$
- NLTK averaged-perceptron HMM

3. CRF / Neural (2000s+) [9]

- CRF: discriminative, $P(\text{tags}|\text{words})$
- Arbitrary overlapping features
- BiLSTM-CRF: reads left→right and right→left
- BERT fine-tuned: >97% on WSJ [4]

Viterbi Algorithm (HMM)

Dynamic programming finds the **globally best** tag sequence in $O(n \cdot T^2)$ time, where n = sentence length and T = number of tags. Greedy left-to-right decoding is faster but suboptimal.

Named Entity Recognition (NER)

NER identifies and classifies **named entity spans** in text: PERSON, ORG, GPE, DATE, MONEY, ...

IOB Tagging Scheme:

Apple	CEO	Tim	Cook	announced	results
B-ORG	O	B-PER	I-PER	O	O

- **B-** (Begin): first token of entity
- **I-** (Inside): continuation token
- **O** (Outside): not an entity

NER Approaches:

- NLTK: MaxEnt chunker on POS tags
- spaCy: transition-based neural model
- BERT-based: SotA on CoNLL-2003 [4]

Production Tip

NER entity-type distribution is a dataset health check. An unexpected spike in one type usually signals a domain mismatch between training data and inference data.

Evolution of Word Representations

Era	Method	Representation	Key limitation
1950s–90s	BoW / TF-IDF [18]	Sparse, $\|V\|$-dim	No semantic proximity
2000s	LSA / LDA [2]	Dense topics	Static, no OOV
2013	Word2Vec [11]	Dense 100–300-dim	One vector per word
2014	GloVe [14]	Global co-occurrence	Same polysemy issue
2016	FastText [3]	Subword n-grams	Still static
2018+	BERT / GPT [4]	Contextual vectors	Computationally heavy

Word2Vec — CBOW and Skip-gram [11]

CBOW

Predicts **centre** word from surrounding context.

Context: [w(t-2), w(t-1), w(t+1), w(t+2)] $\longrightarrow$ **predict:** w(t)

Average context embeddings before hidden layer. Faster; better for frequent words.

Skip-gram

Predicts **context** words from centre word.

Centre: w(t) $\longrightarrow$ **predict:** [w(t-2)...w(t+2)]

One centre generates multiple training pairs. Slower; better for **rare words**.

Self-supervised Objective

Neither architecture needs human labels. Both solve a **fake classification task** (predicting masked neighbours). The weight matrix that emerges from training *is* the word embedding matrix — geometry encodes semantics: $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$

Cosine Similarity

Word2Vec stores each word as a dense vector $\mathbf{v} \in \mathbb{R}^d$. Semantic similarity is measured by:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|} \in [-1, +1]$$

Interpretation:

- $+1$ — identical direction (synonyms)
- 0 — orthogonal (unrelated)
- -1 — opposite (antonyms)

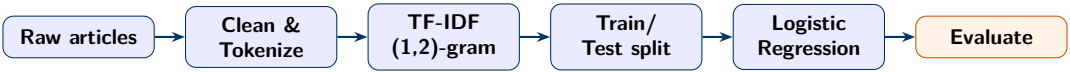
Example pairs (Reuters):

Pair	Cosine
oil — gas	≈ 0.82
bank — stock	≈ 0.71
oil — president	≈ 0.18

PCA Caveat

Projecting 100-dim vectors to 2D via PCA is **lossy**. Clusters visible in 2D are real, but close pairs in 2D may be distant in 100-dim space. Always verify similarity with `model.wv.similarity(w1,w2)` in full dimension.

End-to-End Pipeline — BBC News Classification



Parameter	Why
`min_df=3`	Drop hapax legomena; reduces noise
`ngram_range=(1,2)`	Bigrams capture "stock market", "prime minister"
`sublinear_tf`	Replaces f with $1 + \log f$; compresses outlier counts
`C=5`	Weaker regularisation; high-dim TF-IDF space is noisy
5-fold CV	Estimates generalisation before touching held-out test set

Result

98.65% accuracy on BBC News (5 categories, ≈2,225 articles). Expected — categories are lexically distinct. Real-world noisy data will score lower.

Reading Classification Metrics

Precision, Recall, F1

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P + R}$$

- High P: few false alarms
- High R: few missed instances
- F_1: harmonic mean — punishes extreme imbalance between P and R
- Use **macro-F1** when class sizes differ

Confusion Matrix

- Rows = actual class
- Columns = predicted class
- Diagonal = correct predictions
- Off-diagonal reveals systematic confusion (e.g. tech $\leftrightarrow$ business share vocabulary)

Top Features Per Class:

Logistic Regression coefficients map back to TF-IDF feature names.
High-coefficient terms validate topic-meaningful learning and flag potential data leakage.

Web Scraping with BeautifulSoup

```
import requests
from bs4 import BeautifulSoup
import time

def scrape_wiki(topic):
    url =
        f"https://en.wikipedia.org/wiki/{topic}"
    r = requests.get(url, timeout=10)
    if r.status_code != 200:
        return None
    soup = BeautifulSoup(r.text,
        'html.parser')
    paras = soup.find_all('p')
    text  = '
        '.join(p.get_text(strip=True)
                    for p in paras[:5])
    time.sleep(1)    # politeness delay
    return text
```

Key steps:

- `requests.get()` — HTTP GET, check status 200
- `BeautifulSoup` — parse HTML tree
- `find_all('p')` — extract paragraphs
- `get_text()` — strip all HTML tags
- `time.sleep(1)` — politeness; avoid IP bans

Ethics & Legality

Always check `robots.txt` before scraping. Many sites prohibit automated access. Use official APIs where available.

Session Summary

Core concepts covered:

1. NLP definition, challenges, applications
2. Text Analytics lifecycle (iterative pipeline)
3. NLTK vs spaCy — when to use which
4. Pre-processing: tokenize → normalise → stem/lemmatize → stopwords
5. Feature extraction: BoW → TF-IDF (with formula derivation)
6. N-grams & bigram language models
7. POS tagging: HMM, CRF, BiLSTM-CRF
8. NER: IOB scheme, NLTK vs spaCy
9. Word2Vec: CBOW vs Skip-gram, cosine similarity
10. Text classification: full BBC pipeline, metrics

Key takeaways:

- TF-IDF is a soft stopword filter built into the maths
- Stemming = speed; Lemmatization = accuracy
- Tokenizer choice matters more than most people think
- Word2Vec learns from context, not labels
- 98% accuracy on BBC is easy; real NLP is harder

Next session:

Advanced vectorisation — GloVe, FastText, subword models, contextual embeddings (ELMo, BERT).

Further Reading & Online Resources

Textbooks

- Jurafsky & Martin — *Speech and Language Processing* (3rd ed., free online) [8]
- Manning & Schütze — *Foundations of Statistical NLP* [10]
- Goldberg — *Neural Network Methods for NLP* [5]
- Bird et al. — *Natural Language Processing with Python* [1]

Classic Papers

- Porter (1980) — Suffix-stripping stemmer [15]
- Miller (1995) — WordNet [12]
- Jones (1972) — IDF weighting [7]
- Shannon (1948) — Mathematical theory of communication [19]

Modern Papers

- Mikolov et al. (2013) — Word2Vec [11]
- Pennington et al. (2014) — GloVe [14]
- Bojanowski et al. (2017) — FastText [3]
- Devlin et al. (2019) — BERT [4]
- Lafferty et al. (2001) — CRFs [9]

Online

- SLP3 draft (Stanford)
- HuggingFace NLP Course
- spaCy documentation
- NLTK Book (free)
- Mikolov's Word2Vec code

References I

S. Bird, E. Klein, and E. Loper.
Natural Language Processing with Python.
O'Reilly Media, 2009.
Free online at `https://www.nltk.org/book/`.

D. M. Blei, A. Y. Ng, and M. I. Jordan.
Latent Dirichlet allocation.
Journal of Machine Learning Research, 3:993–1022, 2003.

P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov.
Enriching word vectors with subword information.
Transactions of the Association for Computational Linguistics, 5:135–146, 2017.

J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova.
BERT: Pre-training of deep bidirectional transformers for language understanding.
In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

Y. Goldberg.
Neural Network Methods for Natural Language Processing.
Morgan & Claypool, 2017.

M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd.
spaCy: Industrial-strength natural language processing in python, 2020.

References II

K. S. Jones.
A statistical interpretation of term specificity and its application in retrieval.
Journal of Documentation, 28(1):11–21, 1972.

D. Jurafsky and J. H. Martin.
Speech and Language Processing.
3rd (draft) edition, 2024.
Available at `https://web.stanford.edu/~jurafsky/slp3/`.

J. D. Lafferty, A. McCallum, and F. C. N. Pereira.
Conditional random fields: Probabilistic models for segmenting and labeling sequence data.
In *Proceedings of the 18th ICML*, pages 282–289, 2001.

C. D. Manning and H. Schütze.
Foundations of Statistical Natural Language Processing.
MIT Press, 1999.

T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean.
Distributed representations of words and phrases and their compositionality.
In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013.

G. A. Miller.
WordNet: A lexical database for English.
Communications of the ACM, 38(11):39–41, 1995.

References III

OpenAI.
GPT-4 technical report.
Technical report, OpenAI, 2023.
arXiv:2303.08774.

J. Pennington, R. Socher, and C. D. Manning.
GloVe: Global vectors for word representation.
In *Proceedings of EMNLP*, pages 1532–1543, 2014.

M. F. Porter.
An algorithm for suffix stripping.
Program, 14(3):130–137, 1980.

L. R. Rabiner.
A tutorial on hidden Markov models and selected applications in speech recognition.
Proceedings of the IEEE, 77(2):257–286, 1989.

G. Salton and C. Buckley.
Term-Weighting Approaches in Automatic Text Retrieval, volume 24.
1988.

G. Salton, A. Wong, and C. S. Yang.
A vector space model for automatic indexing.
Communications of the ACM, 18(11):613–620, 1975.

References IV

C. E. Shannon.
A mathematical theory of communication.
The Bell System Technical Journal, 27:379–423, 623–656, 1948.